

PART F: PROJECT REPORT

This project demonstrates the application of data structures, sorting algorithms, algorithm analysis, and Java programming in the development of an integrated student service management system.

The project was divided into several parts. Part A focused on the selection and implementation of appropriate data structures, namely queues, singly linked lists, stacks, and arrays. Part B investigated different sorting algorithms, including Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. Part C experimentally compared the sorting algorithms using different input sizes and an almost-sorted dataset. Part D combined the data structures and sorting algorithms into an integrated Java system. Part E presented the pseudocode for the main operations used in the system.

The purpose of the project was to demonstrate not only how the individual data structures and algorithms work, but also how they can be combined to solve a practical problem.

2. Part A – Data Structures

2.1 Queue

A queue was used to represent students waiting to be served. A queue follows the **First-In, First-Out (FIFO)** principle, meaning that the student who enters the queue first is served first. The project demonstrates a queue containing six student arrivals. After three students are served, the first three students are removed from the front of the queue while the remaining students stay in the queue.

Queue operation

The main queue operations used in the project include:

- **Enqueue** – adds a student to the rear of the queue.
- **Dequeue** – removes and serves the student at the front.
- **Peek/front** – identifies the student currently at the front.
- **isEmpty** – checks whether the queue contains any students.

The queue is appropriate for the student service system because students should normally be attended to in the order in which they arrive.

Queue representation

The queue can be represented conceptually as:

Front → **Student 1** → **Student 2** → **Student 3** → **Student 4** → **Student 5** → **Student 6** ← **Rear**

After serving three students:

Front → **Student 4** → **Student 5** → **Student 6** ← **Rear**

2.2 Singly Linked List

A singly linked list was used to maintain student service records. Each node contains information about a student and a reference to the next node.

The linked-list operations demonstrated in the project include inserting a node, deleting a node, searching for a student, and traversing the list. The submitted Part A work includes diagrams showing insertion and deletion operations.

A simplified representation is:

Head → **Student 1** → **Student 2** → **Student 3** → **NULL**

Each node points to the next node, while the final node points to NULL.

Advantages in the project

The linked list is useful because student records can be added dynamically without requiring the elements to occupy consecutive positions in memory.

The linked list therefore provides a suitable structure for maintaining student service records after students have been served.

2.3 Stack

A stack was used to demonstrate the **Last-In, First-Out (LIFO)** principle.

In a stack, the most recently added item is the first item removed. The project also demonstrates the use of a stack in evaluating postfix expressions. The submitted work provides pseudocode for stack operations and postfix evaluation.

The principal stack operations are:

- **Push** – places an item on top of the stack.
- **Pop** – removes the item from the top.
- **Peek** – examines the top item without removing it.
- **isEmpty** – checks whether the stack contains elements.

A stack is appropriate for postfix expression evaluation because operands can be stored and retrieved according to the LIFO principle.

2.4 Array

An array was used to store numerical values for statistical analysis. The project calculates statistics from the stored values.

The submitted Part A work provides pseudocode for calculating array statistics.

The array provides direct access to its elements using an index and is therefore suitable for storing a fixed collection of service-time values that can later be analysed or sorted.

2.5 Data Structure Selection

The project uses different data structures because each one addresses a different requirement.

Data Structure	Main Use
Queue	Managing students waiting for service
Singly Linked List	Maintaining student service records
Stack	Demonstrating LIFO operations and postfix evaluation
Array	Storing numerical data and calculating statistics

The choices are based on the operations required by each component of the project. The submitted Part A work also provides justifications for the selected data structures.

3. Part B – Sorting Algorithms

Four sorting algorithms were investigated in Part B:

1. Selection Sort
2. Insertion Sort
3. Merge Sort
4. Quick Sort

The project includes traces and explanations showing how each algorithm processes an array.

3.1 Selection Sort

Selection Sort repeatedly searches for the appropriate smallest element from the unsorted portion of the array and places it into its correct position.

The process continues until the entire array has been sorted.

The project uses Selection Sort as one of the algorithms for sorting service-time data.

General process

1. Start at the first position.
2. Find the smallest element in the remaining unsorted section.

3. Swap it with the element at the current position.
4. Move to the next position.
5. Repeat until the array is sorted.

3.2 Insertion Sort

Insertion Sort builds the sorted array one element at a time.

Each new element is compared with the elements already sorted, and elements are shifted when necessary to create the correct position for the new element.

General process

1. Consider the second element as the first element to insert.
2. Compare it with the elements before it.
3. Shift larger elements to the right.
4. Insert the element into its correct position.
5. Repeat until all elements have been processed.

Insertion Sort is particularly useful when the input data is already or almost sorted, which was also investigated in Part C.

3.3 Merge Sort

Merge Sort follows a divide-and-conquer approach.

The array is repeatedly divided into smaller sections until individual elements remain. The smaller sections are then merged together in sorted order.

General process

1. Divide the array into two halves.
2. Recursively divide the halves.
3. Sort the smaller sections.
4. Merge the sorted sections.
5. Continue until the complete array is sorted.

The experiment showed that Merge Sort required relatively few comparisons as the input size increased.

3.4 Quick Sort

Quick Sort also uses a divide-and-conquer approach.

A pivot is selected and the array is partitioned around that pivot. Elements are placed into sections according to their relationship with the pivot, and the sections are then sorted recursively.

The project included Quick Sort as the fourth sorting algorithm used in the experimental comparison.

4. Part C – Algorithm Experiment

4.1 Experimental Purpose

The purpose of Part C was to compare the performance of Selection Sort, Insertion Sort, Merge Sort, and Quick Sort using different input sizes.

The experiment recorded:

- Number of comparisons
- Execution time in nanoseconds

The algorithms were tested using input sizes of:

- 20
- 50
- 100
- 500

An additional experiment was conducted using an almost-sorted array of size 100.

4.2 Experimental Results

Results for different input sizes

Algorithm	n = 20 Comparisons	n = 20 Time (ns)	n = 50 Comparisons	n = 50 Time (ns)
Selection Sort	190	19,078	1,225	50,064
Insertion Sort	113	8,925	582	33,144
Merge Sort	66	25,841	221	48,979
Quick Sort	74	11,193	220	20,061

Larger input sizes

Algorithm	n = 100 Comparisons	n = 100 Time (ns)	n = 500 Comparisons	n = 500 Time (ns)
Selection Sort	4,950	187,704	124,750	5,633,966
Insertion Sort	2,904	167,195	61,981	5,455,002
Merge Sort	533	106,602	3,850	879,178
Quick Sort	640	48,067	5,138	408,801

These measurements are the experimental results recorded in the group's Part C work.

4.3 Almost-Sorted Input

An additional test was performed using an almost-sorted array of size 100.

Algorithm	Comparisons	Execution Time (ns)
-----------	-------------	---------------------

Selection Sort	4,950	178,829
Insertion Sort	103	7,815
Merge Sort	320	87,089
Quick Sort	4,930	408,461

The experimental results show that Insertion Sort required considerably fewer comparisons for the almost-sorted dataset than the other algorithms tested.

4.4 Discussion of Results

The experiment demonstrated that the algorithms behave differently depending on the size and condition of the input data.

Selection Sort produced a predictable increase in comparisons as the input size increased. For 500 elements, it recorded 124,750 comparisons.

Insertion Sort performed particularly well on the almost-sorted input, where it required only 103 comparisons.

Merge Sort produced relatively few comparisons for the larger datasets. At 500 elements, it recorded 3,850 comparisons.

Quick Sort recorded the shortest measured execution time among the four algorithms for the 500-element test in the group's experiment, with a measured time of 408,801 ns.

The experiment therefore demonstrates that algorithm performance cannot be evaluated using only one measurement. Both the number of comparisons and measured execution time provide useful information, and the condition of the input data can also affect performance.

5. Part D – Integrated System

5.1 System Overview

Part D combined the data structures and algorithms into a Java-based student service management system.

The system contains a queue for students waiting to be served, a linked list for student records, and an array for service-time statistics.

The main structures created in the Java system are:

```
Queue waitingQueue = new Queue(20);
```

```
StudentLinkedList recordList = new StudentLinkedList();
```

```
ArrayStatistics dailyStats = new ArrayStatistics(50);
```

The system therefore integrates the concepts developed throughout Parts A–C.

5.2 System Menu

The integrated system contains menu options for performing different operations. The submitted Java work provides menu logic with options numbered 1–11.

The system allows the user to perform operations involving:

- Adding students to the waiting queue
- Serving students
- Maintaining student records
- Recording service times
- Displaying records
- Performing statistical operations
- Sorting service-time values
- Demonstrating the implemented algorithms

5.3 Student Service Process

When a student arrives, the student is added to the waiting queue.

When a student is served:

1. The student is removed from the front of the queue.
2. The student's service record is added to the linked list.
3. The service time is recorded in the array used for statistics.
4. The system can subsequently analyse or sort the recorded service times.

This demonstrates how several data structures can work together within one application.

5.4 Demonstration Data

The submitted Java program includes demonstration student data such as:

- Maria
- Tomas
- Ndapewa
- Simon
- Helena
- Petrus

These records are used to demonstrate the operation of the integrated system.

5.5 Screenshots of the Integrated System

The Queue (A1), Singly Linked List (A2), Array statistics (A4) and the four sorting algorithms (Part B) run inside one program, CampusServiceCentre.java, controlled by an 11-option console menu. Each option calls the matching data structure's own methods, so the classes tested in

Parts A to C are reused unchanged. The postfix Stack (A3) is left out of the menu because the brief treats it as a separate exercise.

Structures used

```
private static Queue waitingQueue = new Queue(20);           // A1
private static StudentLinkedList recordList =                // A2
    new StudentLinkedList();
private static ArrayStatistics dailyStats =                  // A4
    new ArrayStatistics(50);
```

Menu dispatch

```
switch (choice) {
    case 1: handleAddToQueue(); break;           // Queue: enqueue
    case 2: handleServeNext(); break;           // Queue: dequeue
    case 3: waitingQueue.displayQueue(); break; // Queue: display
    case 4: handleAddRecord(); break;           // Linked list: insert
    case 5: recordList.displayStudents(); break; // Linked list: traverse
    case 6: handleSearchRecord(); break;        // Linked list: search
    case 7: handleRemoveRecord(); break;        // Linked list: delete
    case 8: dailyStats.displayStatistics(); break; // Array statistics
    case 9: handleSortServiceTimes(); break;    // Sorting
    case 10: SortExperiment.run(); break;       // Experiment
}
```

Each menu number is wired to one required structure or operation, as the brief's Part D table asks.

Serving a student (where the parts connect)

```
Student served = waitingQueue.dequeue();
if (served == null) {
    System.out.println("The waiting queue is empty. No one to serve.");
    return;
}
System.out.println("Now serving: " + served);
recordList.insertAtEnd(served); // keep record permanent
dailyStats.addServiceTime(served.estimatedServiceTime); // daily stats
```

Serving a student removes them from the front of the queue, saves their record in the linked list, and adds their service time to the array used for the daily statistics.

Sorting the recorded times

```
int[] times = dailyStats.getServiceTimesSnapshot();
int[] toSort = times.clone();
switch (algo) {
    case 1: SelectionSort.sort(toSort, true); break;
    case 2: InsertionSort.sort(toSort, true); break;
    case 3: MergeSort.sort(toSort, true); break;
    case 4: QuickSort.sort(toSort, true); break;
}
```

The user chooses one of the four Part B algorithms, and the array is shown before and after sorting.

Deletion

Option 7 removes a record by position, because the group's deleteStudent() takes a position. The program shows the list first so the user can pick one.

6. Part E – Pseudocode

Part E provides pseudocode for the major operations implemented in the project. The pseudocode represents the logic of the system independently of the Java programming language.

The main pseudocode components cover:

- Queue operations
- Stack operations
- Linked-list operations
- Sorting operations
- Supporting operations used by the integrated system

Pseudocode is useful because it describes the algorithmic steps without depending on Java syntax.

6.1 Queue Pseudocode

The queue pseudocode describes how students are added to and removed from the queue. The general operations include checking whether the queue is full or empty, adding a new student, and removing the student at the front.

6.2 Stack Pseudocode

The stack pseudocode implements the fundamental stack operations required for the project. The principal operations are:

PUSH
POP
PEEK
ISEMPTY

These operations allow the stack to follow the Last-In, First-Out principle.

6.3 Linked-List Pseudocode

The linked-list pseudocode describes how student nodes are created, inserted, searched for, deleted, and traversed.

The linked list begins with a head reference and each node contains a reference to the next node.

6.4 Sorting Pseudocode

Pseudocode was also provided for the sorting algorithms used in the project.

The sorting algorithms covered are:

- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort

The pseudocode provides the logical steps needed to transform an unsorted collection of values into sorted order.

7. Relationship Between the Parts

The different parts of the project are connected and build upon one another.

PART A

Data Structures



PART B

Sorting Algorithms



PART C

Experimental Analysis



PART D

Integrated Java System



PART E

Pseudocode



PART F

Project Report

Part A established the data structures required by the system. Part B introduced the sorting algorithms. Part C tested and compared their performance. Part D integrated the concepts into a Java application, while Part E documented the algorithmic logic using pseudocode. Part F brings these components together into one project report.

8. GitHub Repository

The GitHub repository containing the Java project should be provided in this section.

GitHub Repository Link:

[<https://github.com/Senor-223118648/DSA-Mini-Project>]

9. Conclusion

The project demonstrated the practical application of fundamental data structures and sorting algorithms in a Java-based student service management system.

The project used a queue to manage students waiting for service, a singly linked list to maintain student records, a stack to demonstrate LIFO operations and postfix evaluation, and arrays to store numerical data for statistical analysis.

Four sorting algorithms were investigated: Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. The experimental results demonstrated that their performance varies according to input size and the condition of the data. In particular, the almost-sorted experiment showed that Insertion Sort required only 103 comparisons, while Selection Sort required 4,950 comparisons for the same input size.

The integrated Java system demonstrated how the individual data structures and algorithms can be combined to address a practical student service scenario. The project also provided pseudocode to describe the logic of the implemented operations independently of the Java programming language.

Overall, the project provided practical experience in selecting data structures, implementing algorithms, analysing their performance, and integrating them into a functional Java application.